

StepUp CRM — মডিউলভিত্তিক কার্যনীতি

একটিই Laravel অ্যাপ দুটো কাজ করে: Inertia দিয়ে React পেজ render করে, আর `/v1/*` JSON API সার্ভ করে। পেজ কোনো ডেটা বহন করে না — React mount হয়ে নিজেই API থেকে ডেটা আনে। প্রতিটি লেখার প্রবাহ একই: **Route → Controller → FormRequest → Service → Model → Resource**; বহু-ধাপের কাজ Action-এ, আর Action চালু হয় Observer থেকে।

মডিউল ও তাদের কার্যনীতি

#	মডিউল	কার্যনীতি (এক লাইনে)
1	Staff Authentication	<code>/admin/login</code> → <code>web</code> guard session; একই session cookie দিয়েই <code>/v1</code> API চলে (<code>statefulApi</code>), ব্যর্থ চেষ্টাও activity log-এ ওঠে।
2	Client Portal Auth	<code>/login</code> → <code>client-web</code> guard; <code>is_active</code> না হলে ঢুকতে দেয় না, আর <code>EnsureClientAudience</code> staff session-কে client API-তে ঢুকতে বাধা দেয়।
3	Password Reset	staff ও client-এর টোকেন সম্পূর্ণ আলাদা টেবিলে; staff-এর ক্ষেত্রে কেবল super-admin পারে; মেইল queue-তে যায়।
4	RBAC	spatie/laravel-permission — ৫৪টি permission, ৫টি seeded role; <code>Gate::before</code> দিয়ে super-admin সব পাশ করে; আসল প্রহরী route/controller middleware, sidebar কেবল লুকায়।
5	Access Provisioning	admin client-কে password দিয়ে portal login দেয়, revoke করলে সব token মুছে random ৬৪-অক্ষরের password বসে; staff অ্যাকাউন্টও এখান থেকে তৈরি হয়।
6	User	login পরিচয় — নাম, ইমেইল, ফোন, ছবি, password ও একটি role; চাকরির তথ্য এখানে নয়।
7	Company	সর্বোচ্চ সাংগঠনিক একক; Department/Team/Employee/Project সবই এর অধীনে, soft delete ও FK restrict।
8	Department	company-এর ভেতরে unique; Project-এর department হাতে দেওয়া যায় না, team থেকে আসে।
9	Designation	কোম্পানি-নিরপেক্ষ global পদবি তালিকা; একমাত্র master যেটি soft delete করে না।
10	Employee	একজন user-এর একটি company-তে চাকরির রেকর্ড; <code>unique(user, company)</code> ; নাম/ইমেইল/ছবি <code>users</code> থেকে read-through আসে।
11	Team	company+department-এর ভেতরে দল; FormRequest জোর করে ঠিক একজন leader রাখে; member <code>sync()</code> দিয়ে বসে।
12	Client	portal-এ login করতে পারা গ্রাহক; CSV import ২০০ সারির chunk-এ queue job-এ চলে, বিদ্যমান/ডুপ্লিকেট সারি skip হয়।
13	Project	চুক্তি = team + marketer + client; <code>end_date</code> ও <code>target_deadline</code> derived, <code>amount_due</code> database generated column।
14	Milestone Engine	sales target এক-মাসের milestone-এ ভাগ; report তার <code>week_start</code> যে milestone-এ পড়ে সেখানে জমে; <code>ratio ≥ 0.9</code> → On Track, <code>≥ 0.6</code> → At Risk, নয়তো Off Track।

#	মডিউল	কার্যনীতি (এক লাইনে)
15	Weekly Sales Report	বিক্রি সিস্টেমে ঢোকান একমাত্র পথ; সপ্তাহ ওভারল্যাপ করলে প্রত্যাখ্যাত; save হলেই observer milestone ও health হালনাগাদ করে।
16	Project Notes	কাজের ব্যক্তিগত নোট — তালিকায় কেবল নিজের লেখা নোটই আসে।
17	Dashboard	permission অনুযায়ী section বসে (overview / finance / employee / teams); health সংরক্ষিত column থেকে নয়, প্রতিবার নতুন করে গণনা হয় যাতে list পেজের সাথে মেলে।
18	Client Portal	সম্পূর্ণ read-only; query আগেই <code>client_id</code> দিয়ে সীমিত, তাই অন্যের ডেটা কখনো পথেই আসে না।
19	Profile	নাম/ইমেইল/ছবি/password — staff ও client একই endpoint ব্যবহার করে, password ফাঁকা রাখলে অপরিবর্তিত থাকে।
20	Notification	<code>notifications</code> টেবিল + bell; এখন পর্যন্ত লেখে দুটি উৎস — client CSV import আর payment (received / due / overdue)।
21	Activity Log	কাস্টম audit — কে (causer), কী (description), কার উপর (subject); মুছে পাবে কেবল super-admin।
22	CC Agent Roster	কে queue-তে কাজ করবে ও কোন company-র হয়ে; roster-এ active row না থাকলে permission থাকলেও ৪০৩।
23	CC New Orders / Pick	queue = <code>pending</code> + কারো হাতে নেই; partial unique index দুজনের দ্বন্দ্ব database-এই ঠেকায়; একজন সর্বোচ্চ ৫০টি ধরে রাখতে পারে।
24	CC Status Outcome	agent কেবল approve/cancel/follow-up দিতে পারে; লেখা হয় আগে boneek, পরে CRM ; follow-up দিলে pick খোলা থাকে।
25	CC Order Editing	লেখার আগে দুটি প্রশ্ন — agent কি order ধরে আছে, আর merchant-এর নিয়মে এখনো সম্পাদনাযোগ্য কি; দাম, ছাড়, VAT ও stock-এর নিয়ম inventory থেকে হুবহু ported।
26	CC Courier Score	বাইরের API, ফল cache-এ রাখা; API বন্ধ থাকলে চুপচাপ খালি ফেরে, call center থামে না।
27	CC Reconciler	প্রতি ১০ মিনিটে চলে — merchant নিজে বদলে ফেলা order-এর pick <code>stale</code> করে বন্ধ করে দেয়।
28	System Monitoring	<code>storage/logs</code> -এর Log Viewer; আলাদা permission, seed অনুযায়ী কেবল super-admin।
29	Settings Hub	<code>/admin/settings</code> ব্যবহারকারীর অনুমতিতে খোলা প্রথম section-এ পাঠায়, কোনোটিই না থাকলে ৪০৩।
30	Payment Collection	admin/staff project-এর বিপরীতে টাকা তোলে; এক transaction-এ payment + history + project amount, commit-এর পরে notification, invoice ও মেইল।
31	Payment Notification	client দুই জায়গায় খবর পায় — নিজস্ব <code>payment_notifications</code> তালিকা আর bell-এর <code>notifications</code> ; reminder scheduler থেকে, received payment থেকে।

Payment Collection ও তার Notification

টাকা তোলার প্রবাহ

POST /v1/projects/{project}/payments — কেবল super-admin বা manage payments অনুমতিধারী, মিনিটে ১০ বার (throttle:10,1)।

- StorePaymentRequest** — payment_type অনুযায়ী আলাদা নিয়ম: bank_transfer-এ ব্যাংকের নাম, cheque-এ cheque number, mobile_wallet-এ bkash/nagad/rocket। সবার জন্য — amount ≤ project-এর package_amount, payment_date ভবিষ্যতের নয়, next_payment_date অবশ্যই ভবিষ্যতের, proof ≤ ৫ MB (ছবি বা PDF)।
- proof** থাকলে ImageUploadService আগে ডিস্কে রাখে, তারপর service ডাকা হয়।
- PaymentService::createPayment()** — পুরোটা একটি DB::transaction:
 - PaymentGatewayFactory::resolve() → cash / bank_transfer / cheque / card / mobile_wallet / other সবই ManualPaymentGateway (বাইরে কোনো কল যায় না, MANUAL-... id বানায়); কেবল ssl_gateway → SSLCommerzGateway (HTTP)।
 - gateway-এর উত্তর JsonSchemaValidator দিয়ে যাচাই ও sanitize হয়ে payments.gateway_response -এ বসে।
 - payments row → PaymentHistoryService old/new paid amount সহ audit লেখে → project-এর amount_paid, next_payment_date, last_payment_date হালনাগাদ।
 - হিসাব সবসময় ProjectAmountService থেকে: paid = ওই project-এর সব payment-এর যোগফল, due = total - paid। কোথাও আলাদা করে যোগ-বিয়োগ করা হয় না।
- DB::afterCommit** — commit হওয়ার পরেই কেবল তিনটি কাজ: client-কে notification, InvoiceService::generateInvoice() (INV-2026-08-0001 খাঁচে নম্বর), আর SendInvoiceEmailJob queue-তে। transaction ভেঙে গেলে তাই কোনো notification, invoice বা মেইল যায় না — এটাই afterCommit রাখার কারণ।

Notification কোথায় লেখা হয়

উৎস	কী লেখে	কোথায় দেখা যায়
NotificationService::paymentReceived()	payment_notifications row + PaymentReceived (database channel)	client portal-এর তালিকা ও bell
NotificationService::dueReminder()	payment_due_reminder + PaymentDueReminder	একই দুই জায়গা
NotificationService::overdueReminder()	payment_overdue_reminder + PaymentOverdueReminder	একই দুই জায়গা

- Client API: GET /v1/client/payment-notifications (কেবল অপঠিত), PATCH /v1/client/payment-notifications/{id}/read; অন্য client-এর notification ছুঁলে ৪০৩।
- ডুপ্লিকেট ঠেকানো** — reminder-এর ক্ষেত্রে (project_id, notification_type, notification_date) -এর উপর partial unique index, সঙ্গে service-এর আগাম exists() চেক। তাই command দিনে বারবার চললেও একই due date-এর জন্য একবারই যায়। received payment-এ এই চেক নেই (unique: false), কারণ একই দিনে একাধিক payment ওঠা স্বাভাবিক।

- `next_payment_date` না থাকলে reminder তৈরিই হয় না।

Queue ও Scheduler

কে	কোথায়	কখন চলে	কী করে
<code>SendInvoiceEmailJob</code> (queue job)	<code>QUEUE_CONNECTION=database</code> , তাই <code>jobs</code> টেবিল	payment commit-এর পরপরই dispatch	invoice মেইল পাঠিয়ে <code>email_sent_at</code> stamp করে; আগেই stamp থাকলে বা client-এর ইমেইল না থাকলে চুপচাপ ফিরে যায় (double-send হয় না)।
<code>payment:create-reminders</code> (scheduled command)	<code>routes/console.php</code>	প্রতিদিন সকাল ৯টা (Asia/Dhaka), <code>withoutOverlapping</code>	আজ থেকে <code>PAYMENT_NOTIFY_DAYS</code> (ডিফল্ট ৭) দিনের ভেতরে due হলে due reminder, তারিখ পেরিয়ে গেলে overdue reminder; $due \leq 0$ হলে বাদ; ২০০ সারির chunk-এ চলে।

Worker হাতে চালাতে হয় — `php artisan queue:work` না চললে invoice মেইল `jobs` টেবিলেই পড়ে থাকে, আর `php artisan schedule:work` (বা cron) না চললে কোনো reminder-ই যায় না। কোনো ক্ষেত্রেই UI-তে সংকেত আসে না।

স্বয়ংক্রিয় নিয়ম (মানুষ ছাড়াই যা ঘটে)

- **Project তৈরি হলে** — Observer মাসভিত্তিক milestone বানায় ও প্রথম assignment খোলে; target বদলালে milestone নতুন করে তৈরি হয়।
- **Sales report লেখা হলে** — Observer সঙ্গে সঙ্গে milestone-এর achieved ও project-এর health stamp করে (queue নয়, একই request-এ)।
- **Payment লেখা হলে** — commit-এর পরে (`DB::afterCommit`) client-এর notification, invoice তৈরি ও invoice মেইলের queue job — তিনটিই আপনা-আপনি।
- **Queue-তে চলে** — client CSV import, password reset mail ও `SendInvoiceEmailJob`; worker হাতে চালাতে হয়, না চললে কোনো UI সংকেত নেই।
- **Cron-এ চলে** — `call-center:reconcile` প্রতি ১০ মিনিটে, আর `payment:create-reminders` প্রতিদিন সকাল ৯টায় (Asia/Dhaka)।
- **দুই ডাটাবেসে যৌথ transaction নেই** — তাই call center সবসময় আগে boneek-এ লেখে, পরে CRM-এ; উল্টো হলে CRM এমন কাজের দাবি করত যা ঘটেনি।

ERD

সম্পূর্ণ ডায়াগ্রাম: [docs/erd.mmd](#) (উৎস) ও [docs/erd.pdf](#) (রেভার করা)।